

03-收货管理（二）

1. 收货管理（收货）

今日内容及目标：

- 1. 能够完成 执行收货任务
- 2. 能够完成 收货库存变更
- 3. 能够完成微实战（上架相关功能：创建上架任务，查询待上架任务列表，上架，查询上架记录）

今天我们聚焦 **执行收货** 这一核心流程，结合流程图和代码，拆解 "如何把点击收货的操作，转化为系统中任务、明细、入库单的状态同步"。

1.1 执行收货任务

1.1.1 需求分析

入库管理

入库单

收货

收货记录

上架

未选中任何数据

☐	任务号	任务类型	任务状态	入库单号	仓库名称	货主	商品	数量	完成数量	操作
☐	TSK2025120900003	收货任务	已创建	ASN202512090002	中原01号仓库	千途通物流	娃哈哈纯净水	100	0	收货 更多
☐	TSK2025120900004	收货任务	已创建	ASN202512090002	中原01号仓库	千途通物流	红牛	150	0	收货 更多

收货

任务号: TSK2025120900003

任务类型: 收货任务

任务状态: 已创建

货主: 千途通物流

商品: 娃哈哈纯净水

总数量: 100

完成数量: 0

* 收货数量: 请输入收货数量

* 库存属性: 请选择库存属性

所属仓库: 中原01号仓库

* 储位编码: 请选择储位编码

批次号: 请输入批次号

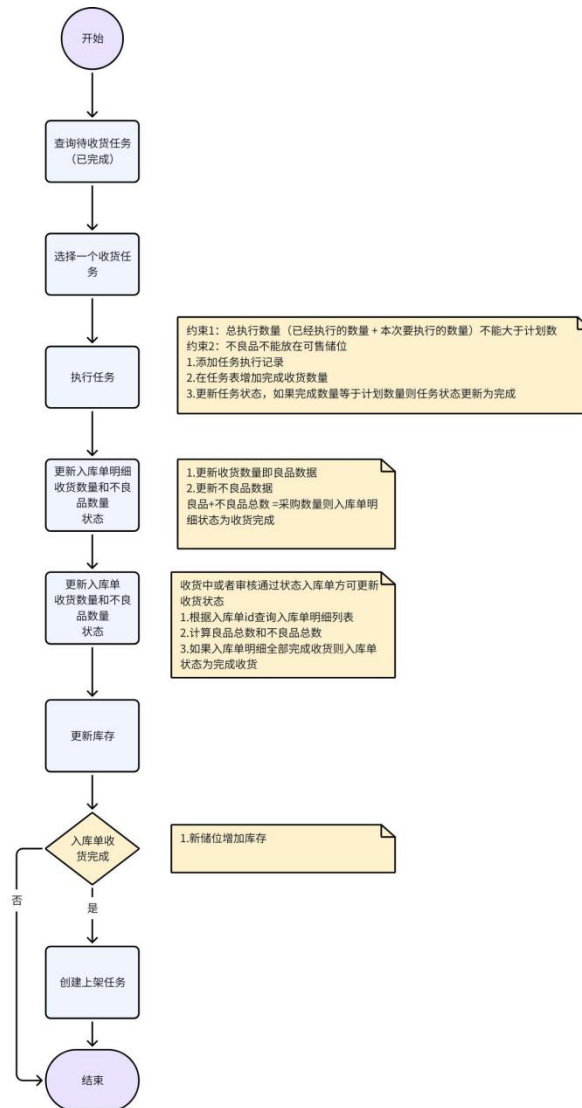
* 收货日期: 请选择收货日期

保质期到期: 请选择保质期到期日

取消 确认

- 记录收货动作：每次收货都要存 "收了多少、是良品还是不良品"；
- 更新任务进度：任务的 "已完成数量" 累加，收完则标记任务为 "已完成"；
- 同步上游状态：入库单明细、入库单的数量 / 状态要跟着变，收完则触发 "创建上架任务"

下方为收货具体流程图：



1.1.2 Controller 层

org.jeecg.modules.wms.inorder.controller.ReceiveTasksController

TypeScript

代码块

```

@PostMapping("/addRecords")
public Result<String> addRecords(@RequestBody WmsTasksRecords wmsTasksRecords) {

```

```

// 前端传的"id"实际是任务 ID，要转存到 taskId 字段
String taskId = wmsTasksRecords.getId();
wmsTasksRecords.setTaskId(taskId);
wmsTasksRecords.setId(null); // 清空 ID，避免覆盖已有记录
wmsTasksService.receive(wmsTasksRecords);
return Result.OK("添加成功!");
}

```

1.1.3 Service 层

org.jeecg.modules.wms.wmstask.service.IWmsTasksService

```

Java
public interface IWmsTasksService extends IService<WmsTasks> {

    /**
     * 收货
     * @param wmsTasksRecords
     */
    void receive(WmsTasksRecords wmsTasksRecords);
}

```

1.1.4 Service 实现类层

org.jeecg.modules.wms.wmstask.service.impl.WmsTasksServiceImpl

receive 方法（总控流程）

```

Java
@Transactional(rollbackFor = Exception.class)
public void receive(WmsTasksRecords wmsTasksRecords) {
    // 步骤 1: 执行收货任务（记录+更新任务）
    execute(wmsTasksRecords);
    // 步骤 2: 更新入库单明细的数量和状态

    stockInOrderItemsService.updateReceivedStatus(wmsTasks.getStockInOrderItemId());
    // 步骤 3: 更新入库单的数量和状态
    String inOrderStatus =
    stockInOrdersService.updateReceivedStatus(wmsTasks.getStockInOrderId());
}

```

```

// 步骤 4: todo 更新库存
// 步骤 5: 若入库单收完, 创建上架任务 (todo 后续实现)

if(WarehouseDictEnum.INBOUND_RECEIVED.getCode().equals(inOrderStat
us)){
    createPutawayTask(wmsTasksRecords.getStockInOrderId());
}
}

```

步骤 1: 执行收货任务 (记录+更新任务): execute 方法

Java

代码块

```

@Transactional(rollbackFor = Exception.class)
private void execute(WmsTasksRecords wmsTasksRecords) {
    //约束 1: 总执行数量 (已经执行的数量 + 本次要执行的数量) 不能大于
    预期计划数
    // 任务对象
    String taskId = wmsTasksRecords.getTaskId();
    WmsTasks wmsTasks = this.getById(taskId);

    // 本次要执行的数量
    Integer execQuantity = wmsTasksRecords.getExecQuantity();
    // 预期收货数量
    Integer expectedQuantity = wmsTasks.getQuantity();
    // 已经执行的数量
    Integer completedQuantity = wmsTasks.getCompletedQuantity();
    // 已经执行的数量 + 本次要执行的数量 不能大于计划数量
    if (completedQuantity + execQuantity > expectedQuantity) {
        throw new JeecgBootException("总执行数量不能大于计划数量
");
    }

    //约束 2: 不良品不能放在可售储位
    // 获取前端传递的库存属性(良品还是不良品)
    String inventoryAttribute =
wmsTasksRecords.getInventoryAttribute();
    // 获取前端传递的储位编码
    String locationCode =
wmsTasksRecords.getTargetLocationCode();
    // 根据储位编码查询对应的储位记录 (对象包含了是否可售)
    // select * from wms_storage_locations where location_code =

```

```
# {locationCode}
    LambdaQueryWrapper<WmsStorageLocations> lambdaQueryWrapper =
new LambdaQueryWrapper<WmsStorageLocations>()
        .eq(WmsStorageLocations::getLocationCode,
locationCode);
    WmsStorageLocations wmsStorageLocations =
wmsStorageLocationsService.getOne(lambdaQueryWrapper);
    // 当前储位是否可售 1 可售 0 不可售
    String isSellable = wmsStorageLocations.getIsSellable();
    if ("1".equals(isSellable) &&
WarehouseDictEnum.INVENTORY_ATTRIBUTE_DEFECTIVE.getCode().equals(i
nventoryAttribute)) {
        throw new JeecgBootException("不能将不良品存放在可售储位
");
    }

    //1.添加任务执行记录
    //拷贝执行任务的信息到任务记录对象、商品 id,任务号,任务类型
    //仓库 id

wmsTasksRecords.setTargetWarehouseId(wmsTasks.getTargetWarehouseId
());
    //入库单 id

wmsTasksRecords.setStockInOrderId(wmsTasks.getStockInOrderId());
    //入库单明细 id

wmsTasksRecords.setStockInOrderItemId(wmsTasks.getStockInOrderItem
Id());
    //波次 id 用于波次管理
    wmsTasksRecords.setWaveOrderId(wmsTasks.getWaveOrderId());
    //波次拣货明细 id 用于波次管理

wmsTasksRecords.setWaveSkuSummaryId(wmsTasks.getWaveSkuSummaryId()
);
    //任务 id
    wmsTasksRecords.setTaskId(wmsTasks.getId());
    //商品 id
    wmsTasksRecords.setProductId(wmsTasks.getProductId());
    //任务编码
    wmsTasksRecords.setTaskNumber(wmsTasks.getTaskNumber());
    //任务类型
    wmsTasksRecords.setTaskType(wmsTasks.getTaskType());
```

```

        //执行时间
        wmsTasksRecords.setOperationTime(new Date());
        //执行人
        wmsTasksRecords.setOperator(wmsTasks.getOperator());
        boolean save = wmsTasksRecordsService.save(wmsTasksRecords);
        if (!save) {
            throw new JeecgBootException("添加任务执行记录失败");
        }

        //2.在任务表增加完成收货数量
        boolean update = update(new LambdaUpdateWrapper<WmsTasks>()
            .eq(WmsTasks::getId, taskId)
            .setSql("completed_quantity = completed_quantity + "
+ execQuantity));
        if (!update) {
            throw new JeecgBootException("更新任务完成数量失败");
        }

        //3.更新任务状态，如果完成数量等于计划数量则任务状态更新为完成
        WmsTasks taskDb = getById(taskId);
        if
        (taskDb.getCompletedQuantity().equals(taskDb.getQuantity())) {
            update(new LambdaUpdateWrapper<WmsTasks>()
                .eq(WmsTasks::getId, taskId)
                .set(WmsTasks::getTaskStatus,
WarehouseDictEnum.TASK_STATUS_COMPLETED));
        }
    }
}

```

步骤 2：更新入库单明细的数量和状态

org.jeecg.modules.wms.inorder.service.IWmsStockInOrderItemsService

```

Java
public interface IWmsStockInOrderItemsService extends
IService<WmsStockInOrderItems> {

    /**
     * 更新收货完成状态
     * @param stockInOrderItemId
     */
    void updateReceivedStatus(String stockInOrderItemId);
}

```

```
}
```

```
org.jeecg.modules.wms.inorder.service.impl.WmsStockInOrderItemsServiceImpl
```

```
Java
```

```
/**
 * @Description: 入库单明细
 * @Author: jeecg-boot
 * @Date: 2025-12-05
 * @Version: V1.0
 */
@Service
public class WmsStockInOrderItemsServiceImpl extends
ServiceImpl<WmsStockInOrderItemsMapper, WmsStockInOrderItems>
implements IWmsStockInOrderItemsService {

    @Autowired
    private IWmsTasksRecordsService wmsTasksRecordsService;

    @Override
    public void updateReceivedStatus(String stockInOrderItemId) {
        // 1. 查明细，查询该明细所有的收货记录
        WmsStockInOrderItems stockInOrderItems =
this.getById(stockInOrderItemId);
        List<WmsTasksRecords> records =
wmsTasksRecordsService.list(new
LambdaQueryWrapper<>(WmsTasksRecords.class)
                .eq(WmsTasksRecords::getStockInOrderItemId,
stockInOrderItemId));

        // 2. 根据收货记录，汇总良品、不良品的总数量
        int badQuantity = records.stream().filter(record ->
record.getInventoryAttribute().equals(WarehouseDictEnum.INVENTORY_
ATTRIBUTE_DEFECTIVE.getCode()))
                .mapToInt(WmsTasksRecords::getExecQuantity).sum();

        // 汇总良品数量
        int goodQuantity = records.stream().filter(record ->
record.getInventoryAttribute().equals(WarehouseDictEnum.INVENTORY_
ATTRIBUTE_GOOD.getCode()))
                .mapToInt(WmsTasksRecords::getExecQuantity).sum();

        // 3. 更新明细数量，如果收完，把状态改成收货完成
```

```

        stockInOrderItems.setReceivedQuantity(goodQuantity);
        stockInOrderItems.setDefectiveQuantity(badQuantity);

        if(stockInOrderItems.getExpectedQuantity().equals(goodQuantity +
        badQuantity)){

            stockInOrderItems.setStatus(WarehouseDictEnum.INBOUND_DETAIL_RECEIVED.getCode());
        }
        this.updateById(stockInOrderItems);

    }
}

```

步骤 3: 更新入库单的数量和状态

org.jeecg.modules.wms.inorder.service.IWmsStockInOrdersService

Java

```

public interface IWmsStockInOrdersService extends
IService<WmsStockInOrders> {

    /**
     * 更新入库单的数量和状态
     */
    String updateReceivedStatus(String stockInOrderId);
}

```

org.jeecg.modules.wms.inorder.service.impl.WmsStockInOrdersServiceImpl

Java

代码块

@Override

```

public String updateReceivedStatus(String stockInOrderId) {

    WmsStockInOrders wmsStockInOrders = getById(stockInOrderId);
    //当前状态只有是收货中或审核通过时方可更新收货状态

    if(!(WarehouseDictEnum.INBOUND_RECEIVING.getCode().equals(wmsStock
    InOrders .getStatus())
        ||
        WarehouseDictEnum.INBOUND_APPROVED.getCode().equals(wmsStockInOrde
    rs .getStatus()))){

```



```

        throw new JeecgBootException("非收货中、审核通过状态入库单不允许更新收货状态");
    }
    // 1.查询该入库单的所有明细
    List<WmsStockInOrderItems> stockInOrderItemsList =
wmsStockInOrderItemsMapper.selectByMainId(stockInOrderId);

    // 2.汇总数量
    int totalCompletedQuantity =
stockInOrderItemsList.stream().mapToInt(WmsStockInOrderItems::getR
eceivedQuantity).sum();
    int totalBadQuantity =
stockInOrderItemsList.stream().mapToInt(WmsStockInOrderItems::getD
efectiveQuantity).sum();

    // anyMatch: 只要有一个明细没收完，入库单就还是"收货中"
    boolean hasUnfinished =
stockInOrderItemsList.stream().anyMatch(item ->
!item.getStatus().equals(WarehouseDictEnum.INBOUND_DETAIL_RECEIVED
.getCode()));
    String status = hasUnfinished ?
WarehouseDictEnum.INBOUND_RECEIVING.getCode() :
WarehouseDictEnum.INBOUND_RECEIVED.getCode();

    // 3.更新入库单的不良品数量，良品数量，状态
    //更新收货总量
    WmsStockInOrders stockInOrdersUpdate =
getById(stockInOrderId);

stockInOrdersUpdate.setTotalReceivedQuantity(totalCompletedQuantit
y);

    //更新不良品数量

stockInOrdersUpdate.setTotalDefectiveQuantity(totalBadQuantity);
stockInOrdersUpdate.setStatus(status);
updateById(stockInOrdersUpdate);
return status;
}

```

步骤 4：若入库单收完，创建上架任务（实战）

1.1.5 测试

1. 单次收货：收 "计划数量" 的一部分，看任务的 "已完成数量" 是否累加，明细的 "已收数量" 是否同步；
2. 多次收货：分 2 次收完计划数量，看任务状态是否变成 "已完成"，明细状态是否变成 "收货完成"；
3. 收完触发上架：收完入库单的所有明细，看是否自动创建上架任务（待完成）。

2. 库存管理

仓储系统的核心是 "账实一致"—— 系统里的库存数据必须和仓库里的实物完全匹配。而库存表就是 "记录实物状态" 的核心载体：

- 它要回答：什么商品、在哪个储位、有多少、是什么状态（可售 / 不可售）、哪个批次。
- 所有操作（收货、上架、出库）最终都会同步到库存表，所以库存表的设计直接决定了系统的准确性。

2.1 库存表设计

库存表：存储仓库中商品在储位的存放情况

库存变更表：存储库存数量变更的记录

wms_inventory /* 库存表 */	
create_by /* 创建人 */	varchar(50)
create_time /* 创建日期 */	datetime
update_by /* 更新人 */	varchar(50)
update_time /* 更新日期 */	datetime
sys_org_code /* 所属部门 */	varchar(64)
product_id /* 商品id */	varchar(32)
location_code /* 储位编码 */	varchar(32)
container_code /* 容器编码 */	varchar(32)
stock_quantity /* 在库数量 */	int
allocated_quantity /* 分配数量 */	int
available_quantity /* 可用数量 */	int
batch_number /* 批号 */	varchar(32)
stock_in_time /* 入库时间 */	datetime
expiry_date /* 保质期到期日 */	date
owner_id /* 货主 */	varchar(32)
is_sellable /* 是否可售 */	varchar(32)
warehouse_id /* 仓库id */	varchar(32)
id	varchar(36)

wms_inventory_trans	
create_by /* 创建人 */	varchar(50)
create_time /* 创建日期 */	datetime
update_by /* 更新人 */	varchar(50)
update_time /* 更新日期 */	datetime
sys_org_code /* 所属部门 */	varchar(64)
product_id /* 商品id */	varchar(32)
location_code /* 储位编码 */	varchar(32)
container_code /* 容器编码 */	varchar(32)
change_quantity /* 变更数量 */	int
transaction_type /* 变更类型 */	varchar(32)
reference_number /* 关联单据号 */	varchar(32)
remarks /* 备注 */	varchar(512)
transaction_time /* 变更时间 */	datetime
batch_number /* 批次号 */	varchar(32)
id	varchar(36)

2.2 表结构分析

SQL

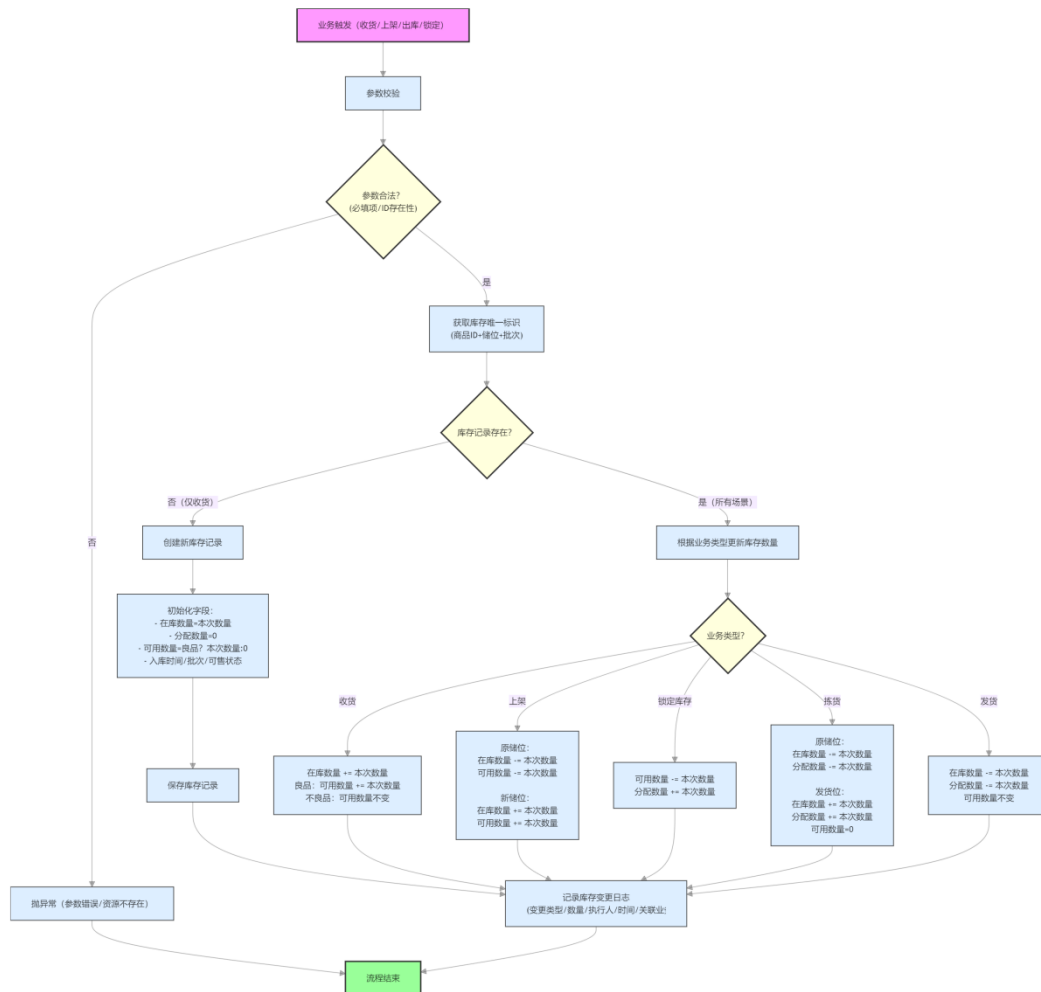
```
create table wms_inventory
(
    id                varchar(36) not null primary key, -- 库存记录 ID (唯一标识)
    create_by         varchar(50) null comment '创建人', -- 谁创建的这条库存记录
    create_time        datetime    null comment '创建日期', -- 记录创建时间
    update_by         varchar(50) null comment '更新人', -- 谁最后更新了这条记录
    update_time        datetime    null comment '更新日期', -- 最后更新时间
    sys_org_code       varchar(64) null comment '所属部门', -- 业务部门 (多部门共用仓库时区分)
    product_id         varchar(32) not null comment '商品 id', -- 关联商品表 (是什么商品)
    location_code       varchar(32) not null comment '储位编码', -- 商品放在哪个储位
    container_code      varchar(32) null comment '容器编码', -- 商品所在的容器 (比如托盘、货架层)
    stock_quantity      int          not null comment '在库数量', -- 这个储位下该商品的实际总数
    allocated_quantity  int          null comment '分配数量', -- 已被订单锁定的数量 (不能再用于其他订单)
    available_quantity  int          null comment '可用数量', -- 能用于销售/出库的数量 (=在库-分配)
    batch_number        varchar(32) null comment '批号 ', -- 商品批次 (食品/医药必须, 区分不同批次)
    stock_in_time       datetime    not null comment '入库时间', -- 商品什么时候进的仓库
    expiry_date         date        null comment '保质期到期日', -- 商品过期时间 (食品/医药必须)
    owner_id            varchar(32) null comment '货主', -- 商品属于哪个客户 (第三方仓储场景)
    is_sellable         varchar(32) null comment '是否可售', -- 1=可
```

售（良品），0=不可售（不良品）

```
warehouse_id      varchar(32) null comment '仓库 id', -- 商品
                    在哪个仓库
constraint wms_inventory_unique01
    unique (product_id, location_code, batch_number) -- 唯一约
束：同一商品+储位+批次只能有一条记录
) comment '库存表';
```

关键字段：

1. `unique (product_id, location_code, batch_number):`
 - 业务意义：同一商品，放在同一个储位，且是同一个批次，只能有一条库存记录。
 - 举例："可乐（商品 ID=1）+A1 储位 + 202501 批次" 是唯一的，不能有两条记录。
2. `stock_quantity/allocated_quantity/available_quantity:`
 - 逻辑关系：`可用数量 = 在库数量 - 分配数量`（良品场景）。
 - 举例：在库 100 件，被订单锁定 30 件（分配数量 = 30），则可用数量 = 70 件。
3. `is_sellable:`
 - 业务意义：区分良品（1）和不良品（0），不良品的 `available_quantity` 永远为 0（不能销售）。



收货是库存的"源头", 我们要实现: 收货完成后, 自动将货物信息同步到库存表。

收货库存变更逻辑: 增加库存时先根据唯一标识查询库存, 如果存在则在更新原有库存记录, 不存在则向库存表增加记录。

2.3 收货存储库存

2.3.1 生成库存表代码

通过 JeecgBoot 代码生成器生成 `wms_inventory` (库存表) 和 `wms_inventory_trans` (库存变更表) 的代码, 包名设为 `inventory`, 并删除无关的 `WmsInventoryTransServiceImpl` 和 `TransController` (后续自定义库存变更逻辑)

代码生成【wms_inventory】

* 代码生成目录:

C:\Users\mrt\Desktop

浏览

页面风格:

经典风格

▼

功能说明:

库存表

✕

* 表名:

wms_inventory

* 实体类名:

WmsInventory

✕

* 包名(小写):

inventory

✕

代码分层样式:

业务分层

▼

* 页面代码:

☒ 封装表单(BasicForm)

☐ 原生表单(a-form)

取消

开始生成

代码生成【wms_inventory_trans】

* 代码生成目录:

C:\Users\mrt\Desktop

浏览

页面风格:

经典风格

▼

功能说明:

库存变更表

✕

* 表名:

wms_inventory_trans

* 实体类名:

WmsInventoryTrans

✕

* 包名(小写):

inventory

✕

代码分层样式:

业务分层

▼

* 页面代码:

☒ 封装表单(BasicForm)

☐ 原生表单(a-form)

取消

开始生成

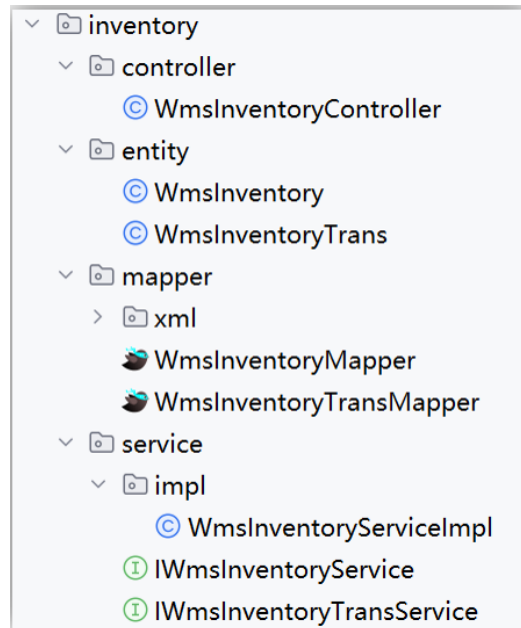
将生成的代码中下边的类删除:

- 删除

`org.jeecg.modules.wms.inventory.service.impl.WmsInventoryTransServiceImpl`

- 删除

org.jeecg.modules.wms.inventory.controller.WmsInventoryTransController



2.3.2 参数实体类

库存变更是个通用操作（收货、上架、出库都需要），所以先定义一个参数类，封装所有需要的信息：

```
Python
package org.jeecg.modules.wms.inventory.vo;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableId;
import com.fasterxml.jackson.annotation.JsonFormat;
import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;
import org.jeecgframework.poi.excel.annotation.Excel;
import org.springframework.format.annotation.DateTimeFormat;

import java.util.Date;

/**
 * @version 1.0
 * @description 库存变更参数类
 */
@Data
public class WmsInventoryTransParam {
```

```
private static final long serialVersionUID = 1L;

/**商品 id*/
@Excel(name = "商品 id", width = 15)
@Schema(description = "商品 id")
private String productId;
/**执行数量*/
@Excel(name = "执行数量", width = 15)
@Schema(description = "执行数量")
private Integer execQuantity;
/**仓库 id*/
@Excel(name = "仓库 id", width = 15)
@Schema(description = "仓库 id")
private java.lang.String warehouseId;
/**来源储位编码*/
@Excel(name = "来源储位编码", width = 15)
@Schema(description = "来源储位编码")
private String sourceLocationCode;
/**目的储位编码*/
@Excel(name = "目的储位编码", width = 15)
@Schema(description = "目的储位编码")
private String targetLocationCode;

/**batchNumber 批次号*/
@Excel(name = "batchNumber 批次号", width = 15)
@Schema(description = "batchNumber 批次号")
private String batchNumber;

@Excel(name = "保质期", width = 20, format = "yyyy-MM-dd")
@JsonFormat(timezone = "GMT+8", pattern = "yyyy-MM-dd")
@DateTimeFormat(pattern="yyyy-MM-dd")
private Date expiryDate;

/**备注*/
@Excel(name = "备注", width = 15)
@Schema(description = "备注")
private String remarks;

/**变更类型*/
@Excel(name = "变更类型", width = 15)
@Schema(description = "变更类型")
```



```

private String transactionType;

/**执行人*/
@Excel(name = "执行人", width = 15)
@Schema(description = "执行人")
private String operator;
/**执行时间*/
@Excel(name = "执行时间", width = 20, format = "yyyy-MM-dd
HH:mm:ss")
@JsonFormat(timezone = "GMT+8", pattern = "yyyy-MM-dd
HH:mm:ss")
@DateTimeFormat(pattern="yyyy-MM-dd HH:mm:ss")
@Schema(description = "执行时间")
private Date operationTime;

/**是否可售 is_sellable*/
@Excel(name = "是否可售 is_sellable", width = 15)
@Schema(description = "是否可售 is_sellable")
private String isSellable;
}

```

2.3.3 根据唯一约束查库存

因为库存表有 `product_id+location_code+batch_number` 的唯一约束，所以每次操作前必须先查 "这个组合是否已存在库存记录"：

```

TypeScript
代码块
// Service 接口定义
public interface IWmsInventoryService extends
IService<WmsInventory> {
    // 根据"商品 ID+储位+批次"查库存
    WmsInventory getInventoryByUniqueKey(String productId, String
locationCode, String batchNumber);
}

// Service 实现
@Service
public class WmsInventoryServiceImpl extends
ServiceImpl<WmsInventoryMapper, WmsInventory> implements
IWmsInventoryService {

```

```

    @Override
    public WmsInventory getInventoryByUniqueKey(String productId,
String locationCode, String batchNumber) {
        // 构造查询条件: 严格匹配"商品 ID+储位+批次"
        LambdaQueryWrapper<WmsInventory> query = new
LambdaQueryWrapper<WmsInventory>()
            .eq(WmsInventory::getProductId, productId)
            .eq(WmsInventory::getLocationCode, locationCode);

        // 批次号可能为空, 空时匹配空字符串 (避免 null 导致查询失败)
        if (StringUtils.isEmpty(batchNumber)) {
            query.eq(WmsInventory::getBatchNumber, batchNumber);
        } else {
            query.eq(WmsInventory::getBatchNumber, "");
        }

        // 返回唯一的库存记录 (因为有唯一约束, 最多一条)
        return getOne(query);
    }
}

```

2.3.4 收货进行库存变更

创建 `WmsInventoryTransByReceiving` 类, 专门处理收货场景的库存变更, 逻辑分为"新增库存"和"更新库存"两种情况:

```

Java
/**
 * @Description: 库存变更表
 * @Author: jeecg-boot
 * @Date: 2026-01-08
 * @Version: V1.0
 */
public interface IWmsInventoryTransService extends
IService<WmsInventoryTrans> {

    /**
     * 库存变更 (通用: 收货、上架、拣货等)
     * @param inventoryTransParam
     */
    public void transfer(WmsInventoryTransParam
inventoryTransParam);
}

```

```
}
```

```
Java
@Service
public class WmsInventoryTransByReceiving extends
ServiceImpl<WmsInventoryTransMapper, WmsInventoryTrans> implements
IWmsInventoryTransService {

    @Autowired
    private IWmsProductsService wmsProductsService;

    @Autowired
    private IWmsStorageLocationsService
wmsStorageLocationsService;

    @Autowired
    private IWmsInventoryService wmsInventoryService;

    @Override
    @Transactional(rollbackFor = Exception.class)
    public void transfer(WmsInventoryTransParam
inventoryTransParam) {
        // ----- 第一步：参数合法性校验 ---
        -----
        // 1. 非空校验：目标储位、商品 ID、执行数量、是否可售必须传
        if
(StringUtils.isEmpty(inventoryTransParam.getTargetLocationCode())
||
StringUtils.isEmpty(inventoryTransParam.getProductId())
|| inventoryTransParam.getExecQuantity() ==
null
||
StringUtils.isEmpty(inventoryTransParam.getIsSellable())) {
            throw new JeecgBootException("目标储位、商品 id、
执行数量、是否可售不能为空");
        }

        // 2. 校验商品和储位的合法性（防止传不存在的 ID）
        WmsProduct product =
wmsProductsService.getById(inventoryTransParam.getProductId());
        if (product == null) {
            throw new JeecgBootException("商品 ID[" +
inventoryTransParam.getProductId() + "]不存在");
        }
    }
}
```

```

    }
    WmsStorageLocations location =
wmsStorageLocationsService.getOne(
        new
LambdaQueryWrapper<WmsStorageLocations>().eq(WmsStorageLocations::
getLocationCode, inventoryTransParam.getTargetLocationCode())
    );
    if (location == null) {
        throw new JeecgBootException("储位编码[" +
inventoryTransParam.getTargetLocationCode() + "]不存在");
    }

    // ----- 第二步：查询是否已有库存 -
    -----
    // 根据唯一约束查询库存
    WmsInventory existingInventory =
wmsInventoryService.getInventoryByUniqueKey(
        inventoryTransParam.getProductId(),
inventoryTransParam.getTargetLocationCode(),
inventoryTransParam.getBatchNumber()
    );

    // ----- 第三步：新增/更新库存 ----
    -----
    if (existingInventory == null) {
        // 情况 1：没有库存 → 新增库存记录
        WmsInventory newInventory = new WmsInventory();
        // 1. 复制参数中的基础信息（商品、仓库、储位、批次等
    )
        BeanUtils.copyProperties(inventoryTransParam,
newInventory);
        // 2. 补充商品关联的货主（从商品表获取）
        newInventory.setOwnerId(product.getOwnerId());
        // 3. 设置库存数量：

newInventory.setStockQuantity(inventoryTransParam.getExecQuantity(
)); // 在库数量=本次收货数量

newInventory.setLocationCode(inventoryTransParam.getTargetLocation
Code());

        //如果储位是可售库存，则设置可用库存为数量

```

```

newInventory.setAvailableQuantity("1".equals(inventoryTransParam.getIsSellable()) ? inventoryTransParam.getExecQuantity() : 0);
newInventory.setAllocatedQuantity(0);
// 4. 设置入库时间和是否可售

newInventory.setIsSellable(inventoryTransParam.getIsSellable());

newInventory.setStockInTime(inventoryTransParam.getOperationTime());
};

// 5. 保存新库存记录
wmsInventoryService.save(newInventory);
} else {
// 情况 2: 已有库存 → 更新库存数量 (累加)
LambdaUpdateWrapper<WmsInventory> updateWrapper
= new LambdaUpdateWrapper<WmsInventory>()
    .eq(WmsInventory::getId,
existingInventory.getId())// 只更新这条库存记录
    // 1. 在库数量累加: 原数量 + 本次收货数量
    .setSql("stock_quantity = stock_quantity
+ " + inventoryTransParam.getExecQuantity())
    // 2. 可用数量: 只有良品才累加, 不良品不更新

.setSql("1".equals(inventoryTransParam.getIsSellable()),
"available_quantity = available_quantity + " +
inventoryTransParam.getExecQuantity());
// 执行更新
wmsInventoryService.update(updateWrapper);
}

// ----- 第四步: 写入库存变更表 -----
-----
// 1. 构建库存变更记录对象
WmsInventoryTrans transRecord = new
WmsInventoryTrans();
// 2. 复制参数中的基础信息 (创建人、商品、储位、批次、变更
类型等)
BeanUtils.copyProperties(inventoryTransParam,
transRecord);
// 3. 补充必要字段 (根据 wms_inventory_trans 表结构)
BeanUtils.copyProperties(inventoryTransParam,
transRecord);
transRecord.setLocationCode(

```

```

inventoryTransParam.getTargetLocationCode());

transRecord.setCreateBy(inventoryTransParam.getOperator()); // 执行人作为创建人
        transRecord.setCreateTime(new Date()); // 当前时间作为创建时间

transRecord.setUpdateBy(inventoryTransParam.getOperator());
        transRecord.setUpdateTime(new Date());

transRecord.setChangeQuantity(inventoryTransParam.getExecQuantity()); // 变更数量=本次执行数量

transRecord.setTransactionTime(inventoryTransParam.getOperationTime()); // 变更时间=执行时间

transRecord.setTransactionType(inventoryTransParam.getTransactionType()); // 类型
        transRecord.setRemarks("收货,向库存新增记录,入库单:" +
inventoryTransParam.getRemarks()); //备注
        // 4. 保存库存变更记录
        this.save(transRecord);
    }
}

```

2.3.5 收货流程中调用库存变更

回到 `WmsTasksServiceImpl` 的 `receive` 方法（收货的核心方法），在收货完成后，触发库存同步：

```

Java
@Service
public class WmsTasksServiceImpl extends
ServiceImpl<WmsTasksMapper, WmsTasks> implements IWmsTasksService
{

    @Autowired
    @Qualifier("wmsInventoryTransByReceiving")
    private IWmsInventoryTransService
wmsInventoryTransByReceiving;

    /**

```

```

    * 收货
    * @param wmsTasksRecords
    */
    @Override
    @Transactional(rollbackFor = Exception.class)
    public void receive(WmsTasksRecords wmsTasksRecords) {
        // 1.执行收货任务（记录 + 更新任务）
        WmsTasks wmsTasks = execute(wmsTasksRecords);
        // 2.更新入库单明细的数量和状态

        stockInOrderItemsService.updateReceivedStatus(wmsTasks.getStockInOrderItemId());
        // 3.更新入库单的数量和状态
        String inOrderStatus =
        stockInOrdersService.updateReceivedStatus(wmsTasks.getStockInOrderId());

        // ----- 新增：同步库存 -----
        -----
        // 4.更新库存
        WmsInventoryTransParam inventoryTransParam = new
        WmsInventoryTransParam();

        inventoryTransParam.setProductId(wmsTasksRecords.getProductId());

        inventoryTransParam.setExecQuantity(wmsTasksRecords.getExecQuantity());

        inventoryTransParam.setWarehouseId(wmsTasksRecords.getTargetWarehouseId());

        inventoryTransParam.setTargetLocationCode(wmsTasksRecords.getTargetLocationCode());

        inventoryTransParam.setBatchNumber(wmsTasksRecords.getBatchNumber());

        inventoryTransParam.setExpiryDate(wmsTasksRecords.getExpiryDate());
        ;

        inventoryTransParam.setTransactionType(WarehouseDictEnum.INVENTORY_RECEIVING.getCode());

        inventoryTransParam.setRemarks("收货,向库存新增记录,入库单:"+wmsTasksRecords.getStockInOrderId());
    }

```

```

        //保质期到期日

inventoryTransParam.setExpiryDate(wmsTasksRecords.getExpiryDate())
;
        //入库时间

inventoryTransParam.setOperationTime(wmsTasksRecords.getOperationTime());

        // 良品→可售（1）；不良品→不可售（0）

inventoryTransParam.setIsSellable(WarehouseDictEnum.INVENTORY_ATTRIBUTE_GOOD.getCode().equals(wmsTasksRecords.getInventoryAttribute())) ? "1" : "0");

wmsInventoryTransByReceiving.transfer(inventoryTransParam);

    }

```

2.3.6 测试

按以下流程测试，观察库存表的变化：

1. 创建并审核入库单：确保入库单状态为 "审核通过"；
2. 创建收货任务：关联入库单，指定执行人；
3. 执行收货：填写收货数量、储位、批次（良品 / 不良品）；
4. 检查库存表：
 - 首次收货：库存表新增一条记录，`stock_quantity` 和 `available_quantity`（良品）等于收货数量；
 - 再次收货同一商品 + 储位 + 批次：`stock_quantity` 和 `available_quantity`（良品）累加；
 - 不良品收货：`available_quantity` 始终为 0。

3. 上架管理(实战)

3.1 任务目标

通过实战开发，掌握仓储系统中 "上架" 功能的完整实现逻辑，理解 "收货暂存→永久储位" 的库存转移机制，

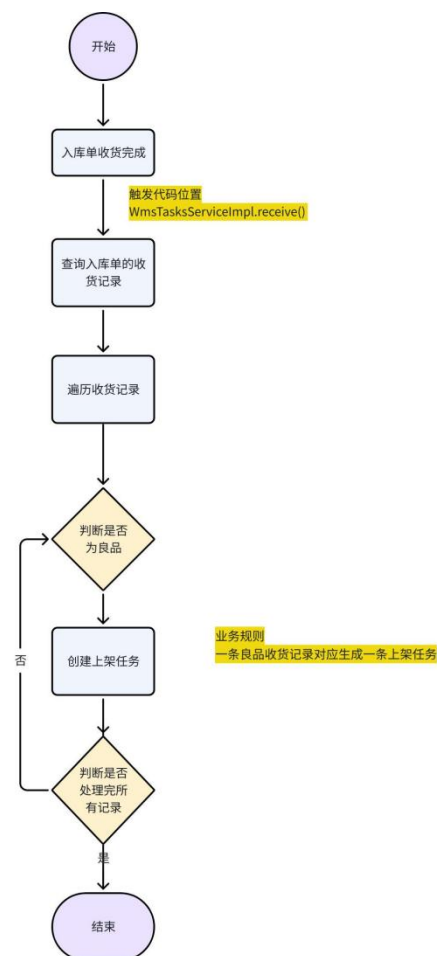
例：将 A01 暂存区的 100 箱饮料转移到 B02 货架，完成后 B02 储位的库存增加 100，A01 暂存区库存清零。

最终实现：

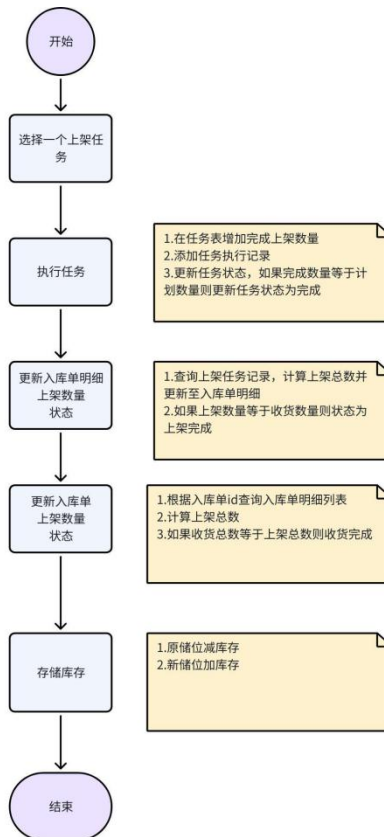
1. 自动创建上架任务（收货完成后）
2. 查询待上架任务列表
3. 执行上架操作（同步库存转移）
4. 查询上架记录

3.2 流程分析

- 创建上架任务



- 上架流程



3.3 创建上架任务

前置准备

拷贝 课程资料-->代码-->初始代码--上架--PutawayTasksController.java

到 org.jeecg.modules.wms.inorder.controller 包下

任务内容:

实现 "收货完成后自动创建上架任务" 的功能, 需满足:

- 仅当入库单状态变为 "收货完成" 时创建
- 仅为 "良品" 创建上架任务 (不良品跳过)
- 每条良品收货记录对应 1 条上架任务

开发提示:

1. 触发时机: 在 WmsTasksServiceImpl 的 receive 方法中, 判断入库单状态为 INBOUND_RECEIVED 时调用创建方法

```

Java
// 参考代码片段
String inOrderStatus =
stockInOrdersService.updateReceivedStatus(stockInOrderId);
if
(WarehouseDictEnum.INBOUND_RECEIVED.getCode().equals(inOrderStatus
)) {
    // 调用创建上架任务的方法
    createPutawayTask(stockInOrderId);
}

```

2. 任务表字段设置：

- `task_type`= 上架（参考枚举 `WarehouseDictEnum.TASK_TYPE_PUTAWAY`）
- `task_status`= 已创建（`TASK_STATUS_CREATED`）
- `source_location_code`= 收货时的暂存储位
- `quantity`= 收货数量（需从收货记录中获取）

检验标准：

收货完成后，查询 `wms_tasks` 表，是否生成符合条件的上架任务记录

124	1999754575498719234	admin	PUTAWAY_TASK	2025-12-13 16:13:39	<null>	<null>
125	1999754575519690753	admin	PUTAWAY_TASK	2025-12-13 16:13:39	<null>	<null>
126	1999754575544856577	admin	PUTAWAY_TASK	2025-12-13 16:13:39	<null>	<null>
127	1999754575565828097	admin	PUTAWAY_TASK	2025-12-13 16:13:39	<null>	<null>
128	1999754575586799618	admin	PUTAWAY_TASK	2025-12-13 16:13:39	<null>	<null>

收货完成后，会自动创建上架任务，任务表中会出现任务类型为上架的记录（几条收货记录就对应几条上架记录），注意：不良品不会创建上架任务

3.4 查询待上架任务列表

任务内容：

开发接口，支持执行人查询自己的待上架任务（状态为 "已创建" 或 "执行中"），需返回：

- 任务 ID、任务编号、商品名称、来源储位名称、待上架数量、入库单号

开发提示：

1. Controller 层：参考收货任务查询，新增 `PutawayTasksController`，提供 `/list` 接口

Java

代码块

```
@Operation(summary="待上架任务查询")
@GetMapping(value = "/list")
public Result<IPage<WmsTasks>> list(WmsTasks wmsTasks,
                                     @RequestParam(name="pageNo",
                                     defaultValue="1") Integer pageNo,
                                     @RequestParam(name="pageSize",
                                     defaultValue="10") Integer pageSize,
                                     HttpServletRequest req) {
    // 1. 关键：设置任务类型为"上架任务"，过滤其他任务

    wmsTasks.setTaskType(WarehouseDictEnum.TASK_TYPE_PUTAWAY.getCode())
    );
    // 2. 设置执行人=当前登录用户（仅查自己的任务）
    LoginUser sysUser = (LoginUser)
    SecurityUtils.getSubject().getPrincipal();
    wmsTasks.setOperator(sysUser.getId());
    // 3. 调用 Service 查询分页数据（复用任务表的通用查询方法）
    IPage<WmsTasks> wmsTasksIPage =
    wmsTasksService.list(wmsTasks, pageNo, pageSize);
    return Result.OK(wmsTasksIPage);
}
```

1. 关联查询：在 Mapper 中编写 SQL，关联 `wms_products`（商品表）和 `wms_storage_locations`（储位表）

检验标准：

调用接口后，能正确返回当前执行人的待上架任务，且包含关联信息

任务号	任务类型	任务状态	仓库名称	货主	商品	数量	完成数量	来源储位编码	操作
TSK2025091100004	上架任务	已创建	中原01号仓库	千途通物流	娃哈哈纯净水	100	0	Z1A-01-01	上架 更多
TSK2025091100002	上架任务	已创建	中原02号仓库	千途通物流	娃哈哈纯净水	100	100	A2-A-01-02	上架 更多

3.5 执行上架操作（核心：库存转移）

任务内容：

开发上架接口，实现 "从来源储位（暂存）转移到目标储位（正式）"，需同步更新：

- 上架任务状态（已完成数量累加）

- 库存表：来源储位扣减，目标储位增加
- 记录库存变更日志

开发提示：

1. 库存转移逻辑：创建 `WmsInventoryTransByPutway` 类，实现 `transfer` 方法

```
TypeScript
@Override
@Transactional
public void transfer(WmsInventoryTransParam param) {
    // 1. 来源储位扣减库存
    deductSourceInventory(param);
    // 2. 目标储位增加库存
    addTargetInventory(param);
    // 3. 记录库存变更日志
    recordTransLog(param);
}
```

2. 扣减来源库存：需校验库存充足 (`stock_quantity >= 本次数量`)

```
Java
private void deductSourceInventory(WmsInventoryTransParam param) {
    WmsInventory source =
wmsInventoryService.getInventoryByUniqueKey(
        param.getProductId(), param.getSourceLocationCode(),
param.getBatchNumber()
    );
    if (source == null || source.getStockQuantity() <
param.getExecQuantity()) {
        throw new RuntimeException("来源储位库存不足");
    }
    // 更新: stock_quantity -= 数量, available_quantity -= 数量
}
```

3. 增加目标库存：若目标储位已有该商品 + 批次，累加数量；否则新增记录

检验标准：

上架后：

- 来源储位库存数量正确减少
- 目标储位库存数量正确增加

- 任务表的 `completed_quantity` 正确累加

3.6 查询上架记录

任务内容：

开发接口，查询某上架任务的所有执行记录，需包含：

- 记录 ID、执行时间、执行人、来源储位、目标储位、执行数量、商品名称

开发提示：

1. 数据来源：从 `wms_tasks_records` 表查询（任务记录），关联商品表和储位表
2. 接口设计：在 `PutawayTasksController` 中新增 `/records` 接口，接收 `wmsTasksRecords` 参数

```
TypeScript
@GetMapping("/records")
public Result<IPage<WmsTasksRecordsVO>> records(WmsTasksRecords
wmsTasksRecords, Integer pageNo, Integer pageSize) {
    IPage<WmsTasksRecordsVO> records =
wmsTasksRecordsService.queryPutawayRecords(taskId, pageNo,
pageSize);
    // 1. 过滤任务类型：仅查询上架任务的记录
    // 2. 分页查询（复用任务记录表的通用分页方法）

    return Result.OK(records);
}
```

检验标准：

调用接口后，能正确返回该任务的所有上架记录，包括每次的执行数量和储位信息

4. 完整测试流程

1. 前置操作：
 - 创建入库单（状态：待审核）→ 审核入库单（状态：审核通过）
 - 创建收货任务 → 执行收货（选择良品，暂存在 A01 储位，数量 100）

2. 验证阶段 1:

- 检查入库单状态是否为 "收货完成"
- 检查 `wms_tasks` 表是否生成上架任务 (`source_location_code=A01`, 数量 100)

3. 验证阶段 2:

- 调用待上架任务列表接口, 确认能查询到步骤 2 生成的任务

4. 验证阶段 3:

- 调用上架接口, 指定目标储位 B01, 执行数量 50
- 检查库存表: A01 储位数量 = 50, B01 储位数量 = 50
- 再次调用上架接口, 执行剩余 50
- 检查任务状态是否变为 "已完成"

5. 验证阶段 4:

- 调用上架记录接口, 确认能查询到 2 条记录 (各 50)